

Code:

```
#include <stdio.h> #include <string.h>

#define MAXN 200005

typedef struct { int pre, suf, max_len; char left_char, right_char; } Node;

Node tree[4 * MAXN]; char s[MAXN];

int max(int a, int b) { return (a > b) ? a : b; }

void pull(int k, int l, int r) { int mid = (l + r) / 2; int lc = 2 * k, rc = 2 * k + 1;

tree[k].left_char = tree[lc].left_char;
tree[k].right_char = tree[rc].right_char;

tree[k].pre = tree[lc].pre;
if (tree[lc].pre == (mid - 1 + 1) && tree[lc].right_char ==
tree[rc].left_char) {
    tree[k].pre += tree[rc].pre;
}

tree[k].suf = tree[rc].suf;
if (tree[rc].suf == (r - mid) && tree[rc].left_char ==
tree[lc].right_char) {
    tree[k].suf += tree[lc].suf;
}

tree[k].max_len = max(tree[lc].max_len, tree[rc].max_len);
if (tree[lc].right_char == tree[rc].left_char) {
    tree[k].max_len = max(tree[k].max_len, tree[lc].suf +
tree[rc].pre);
}

}

void build(int k, int l, int r) { if (l == r) { tree[k].pre = tree[k].suf = tree[k].max_len = 1;
tree[k].left_char = tree[k].right_char = s[l - 1]; return; } int mid = (l + r) / 2; build(2 * k, l,
mid); build(2 * k + 1, mid + 1, r); pull(k, l, r); }
```

```

void update(int k, int l, int r, int pos) { if (l == r) { tree[k].left_char = tree[k].right_char =
(tree[k].left_char == '0' ? '1' : '0'); return; } int mid = (l + r) / 2; if (pos <= mid) update(2 * k, l,
mid, pos); else update(2 * k + 1, mid + 1, r, pos); pull(k, l, r); }

int main() { if (scanf("%s", s) != 1) return 0; int n = strlen(s); int m; if (scanf("%d", &m) != 1)
return 0;

build(1, 1, n);

for (int i = 0; i < m; i++) {
    int x;
    scanf("%d", &x);
    update(1, 1, n, x);
    printf("%d", tree[1].max_len);
    if (i < m - 1) printf(" ");
}
printf("\n");

return 0;

}

```

Input:

```

001011
3
3 2 5

```

Output:

```

4 2 3

```